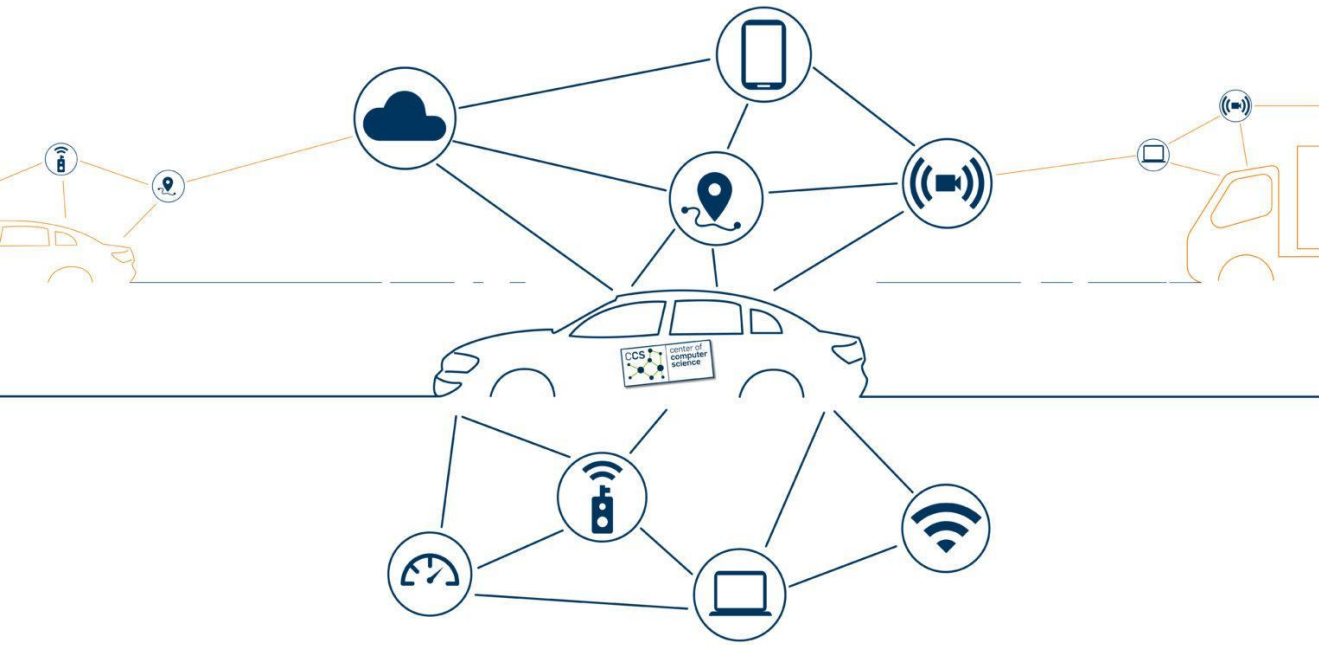




When Missions Go Wrong: A Study of Bugs in Behavior Trees

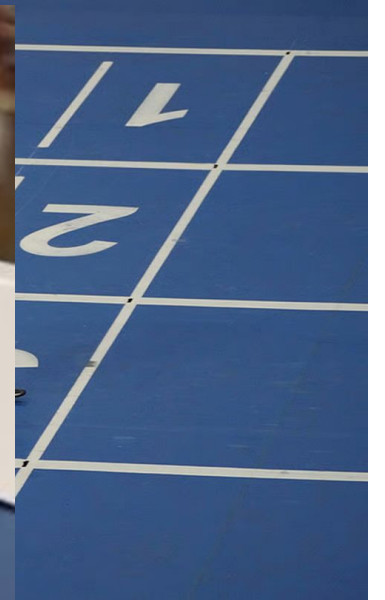
Chair of Software Engineering
Faculty of Computer Science

RUHR
UNIVERSITÄT
BOCHUM

RUB

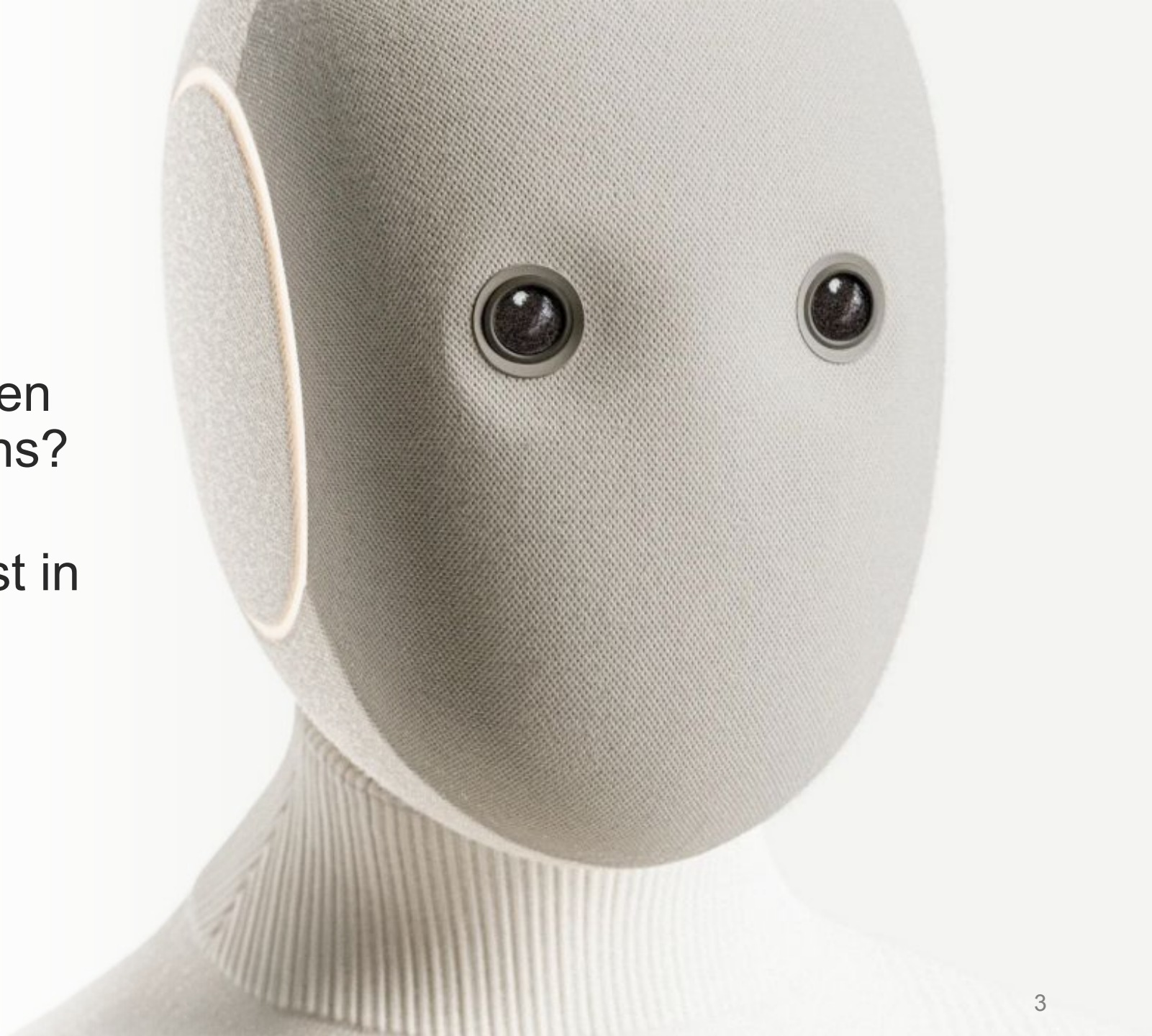
Vicente Romeiro
Supervisor: Prof. Dr. Thorsten Berger

World Humanoid Robot Games SHL



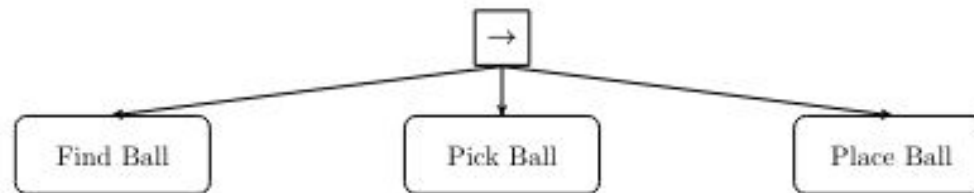
Motivation

- What kinds of faults developers introduce when designing robotic missions?
- How these faults manifest in robotic behaviors?



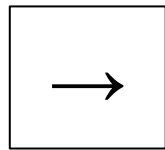
Behavior Trees

- Behavior Trees (BTs) are a model for the control structure of tasks in an autonomous agent
- Commonly used in robotics, AI and video games



(a) A high level BT carrying out a task consisting of first finding, then picking and finally placing a ball.

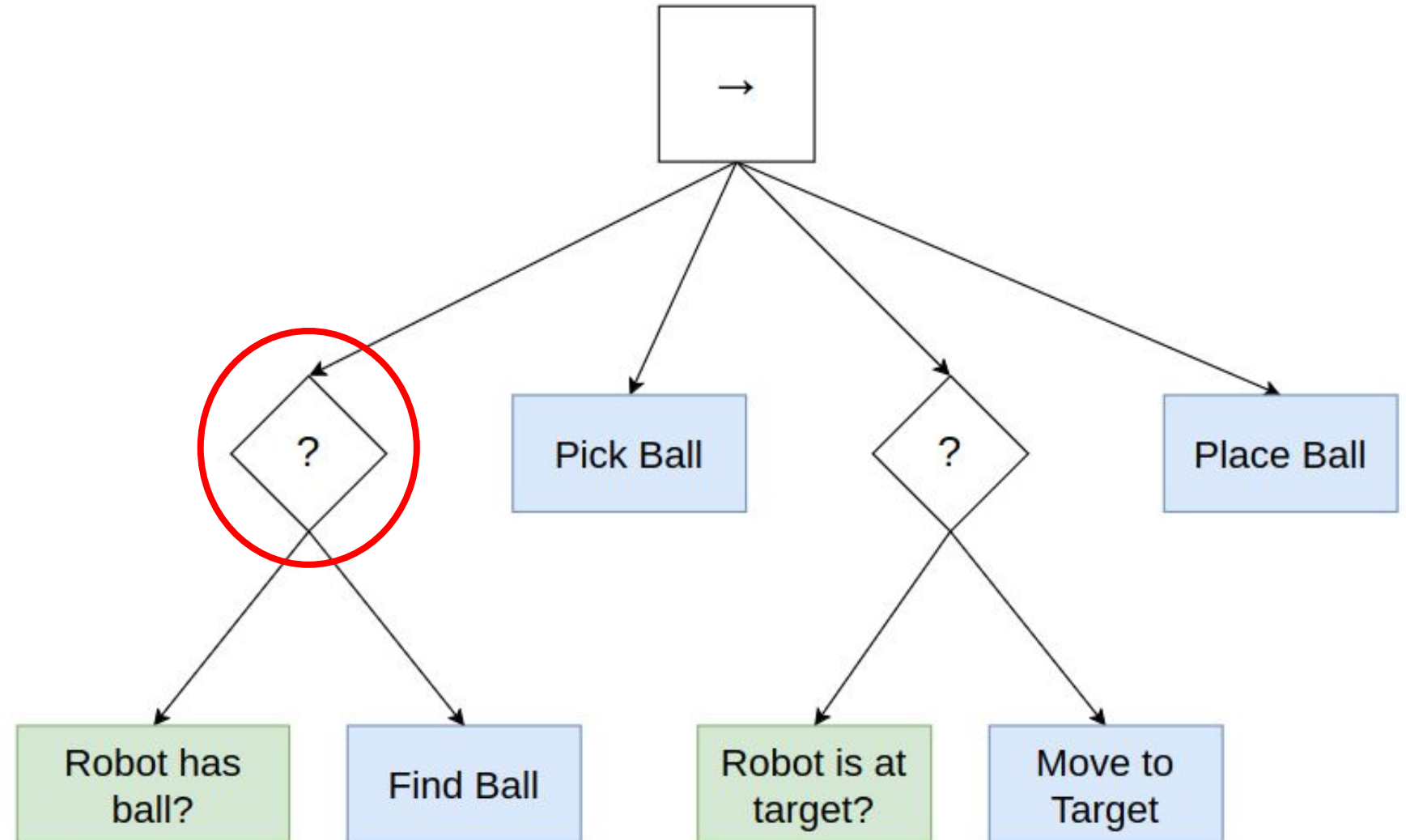
Behavior Trees



Sequence
Executes behaviors sequentially



Fallback
Executes alternative behaviors



Research Problem

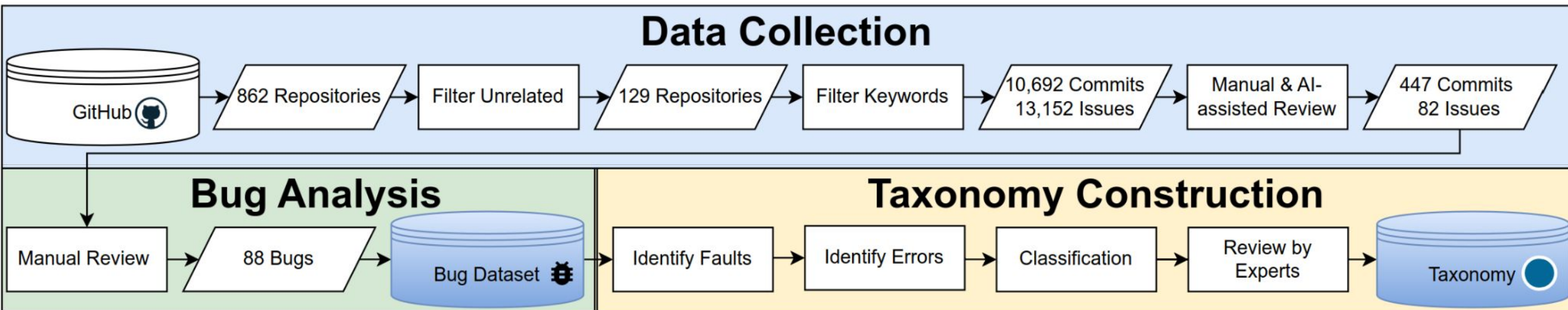
While other works have investigated behavior trees and robotic mission engineering, a study of what happens when these missions go awry is still missing from the literature.

Specifically, our aim was to investigate:

- What bugs occur in Behavior Trees?
 - We extract bugs from open-source repositories and construct a taxonomy to capture recurring patterns in these bugs
- What strategies do developers employ to identify and resolve such bugs?
 - We assess the data and examine the challenges developers face during bug detection and fixing.

Methodology

- We mined 862 open-source repositories on GitHub containing Behavior Trees
- We analyse the bugs encountered and construct a taxonomy around them



Results

We mined 129 repositories from GitHub:

- 10,692 Commits
- 13,152 Issues

From these we performed a manual and an AI-assisted review of the data:

- 447 commits
- 82 issues

Which contained relevant messages that were likely to contain bugs. Finally we obtained:

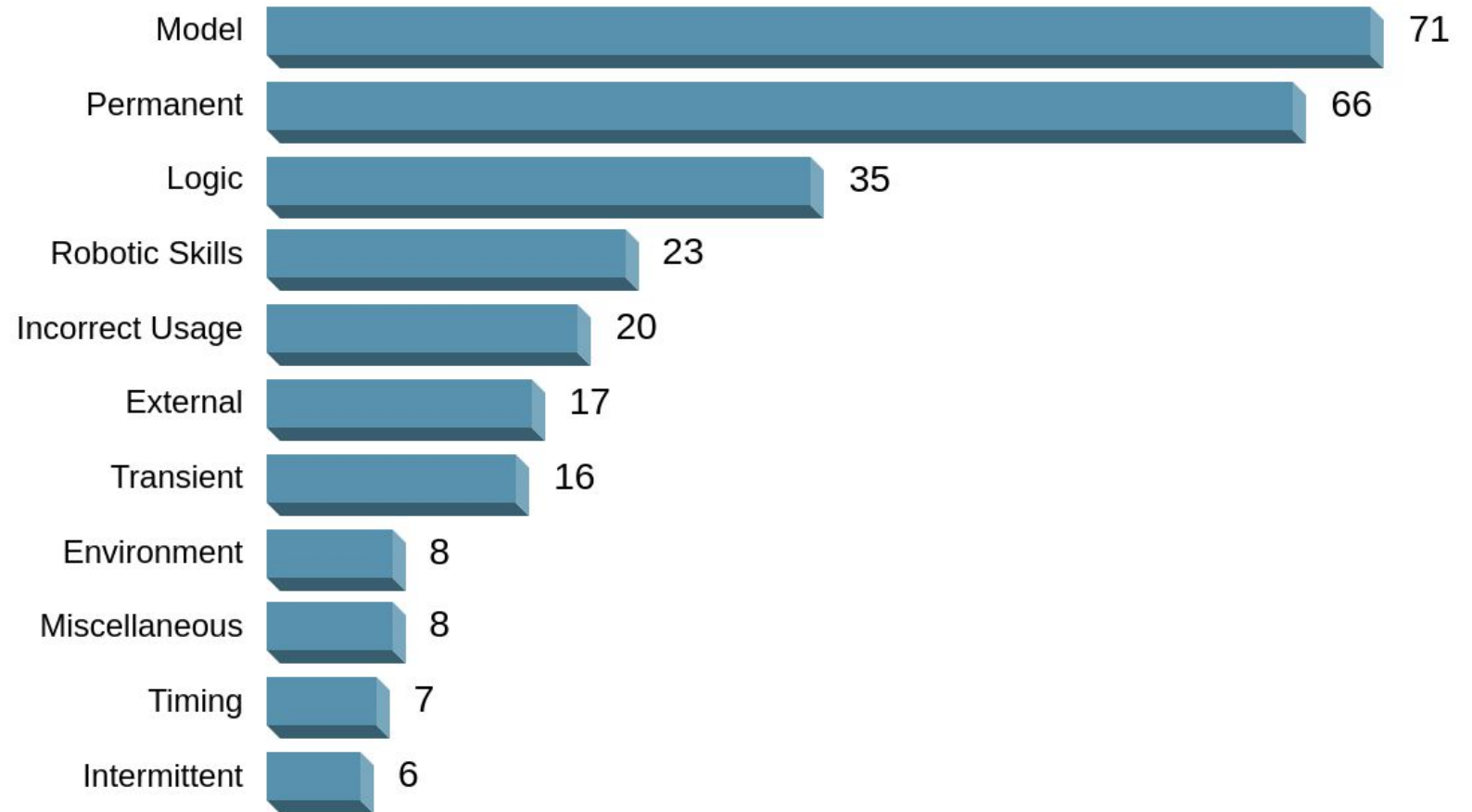
- **88 bugs** related to behaviour trees

Fault Taxonomy

CATEGORY	TYPE	DESCRIPTION	COUNT
Persistence	Permanent	Fault always occurs during execution	66
	Transient	Fault occurs rarely or sporadically	16
	Intermittent	Fault occurs in bounded, predictable intervals	6
Location	Model	Fault occurs directly inside the Behavior Tree model	71
	External	Fault originates in external hardware or software libraries	17
Behavior	Incorrect Usage	Fault caused by incorrect use of Behavior Tree constructs	20
	Timing	Fault caused by incorrect temporal behavior	7
	Logic	Fault caused by incorrect behavioral logic	35
Configuration	Robotic Skills	Fault caused by incorrect configuration of robotic skills (e.g. navigation)	23
	Environment	Fault caused by incorrect map or environment properties read by BT	8
	Miscellaneous	Fault caused by incorrect configuration of other components	8

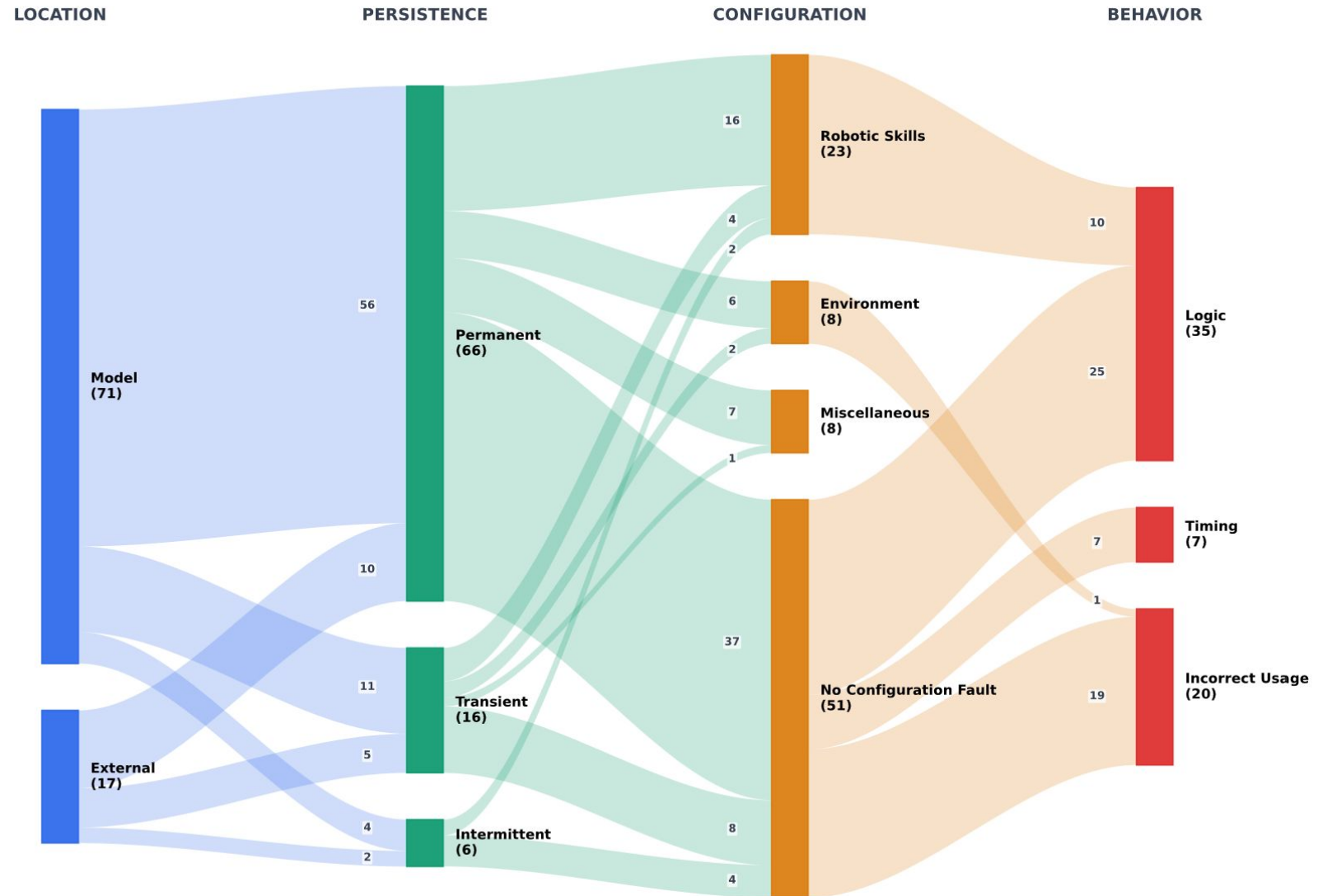
Fault Distribution

- Most faults are Model level (81%)
- Most faults were also Permanent (76%)
- Behavioral faults were caused by:
 - Logic (39%)
 - Incorrect Usage (23%)
 - Timing (8%)
- Configuration Faults were caused by:
 - Robotic Skills (26%)
 - Environment (9%)
 - Others (9%)



Relationship between faults

- Most model faults were also Permanent (80%)
- Most faults were related to behavioral faults (58%)
- All timing faults were unrelated to configuration
- Almost all incorrect usage faults were related to behavioral faults
- Logic faults had the highest correlation with configuration faults



Developer Strategies

Simulation:

Developers rarely identified faults before deployment.

Bugs were mostly identified empirically rather than through static verification.

Manual Reasoning:

Incorrect Usage faults were generally resolved manually,

We found little evidence of automated analyses capable of identifying these issues before execution.

Software Evolution:

Logic faults typically emerged while extending existing missions with new behaviors.

Developers generally fixed them with localised changes to individual nodes

Reactive:

Despite the growing maturity of behavior-tree frameworks, we found little evidence of model-level verification or automated debugging techniques being routinely employed.

Main Takeaways

Modelling Behavior Trees:

The predominance of logic and incorrect usage faults indicate that, while BTs are expressive, their semantics are not always intuitive to developers

Lack of automated tooling:

We found no automated tools for testing or verification being consistently employed by developers

Software Engineering Practices:

Behavior Trees are used as modeling artifacts but are rarely treated with the same rigor as traditional software artifacts in terms of validation, testing, or lifecycle integration

Implications

Developers:

Treating BTs as first-class artifacts would facilitate early detection and reduce the risk of latent faults in the system, thus making a robotic mission safer and more reliable as a whole

Researchers:

- *Further research is needed to investigate what software engineering practices can be best applied in behavior trees*
- *There is a need for empirical studies and automated tooling that focus specifically on behavior trees*

Verification and Validation (V&V):

- *BTs should be treated as first-class artefacts for V&V*
- *Given the importance of BTs for robotics, there is a substantial need for automated testing, formal validation models and runtime verification of properties*

Future Work

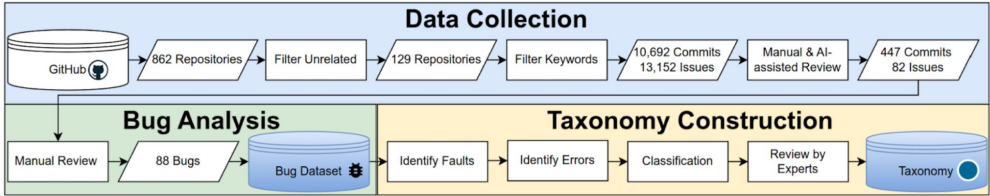
As future work, we plan to:

- Extend this analysis to other behavioral models, such as state machines
- Design a general property specification language for specifying properties over behaviour trees, inspired by the faults identified.
- Create a static verification technique to verify these properties in behaviour trees;

Conclusion

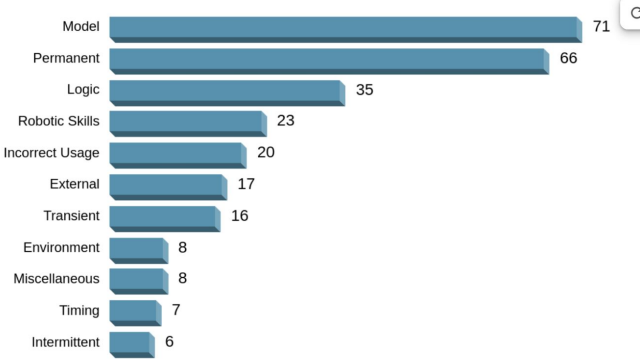
Methodology

- We mined 862 open-source repositories on GitHub containing Behavior Trees
- We analyse the bugs encountered and construct a taxonomy around them



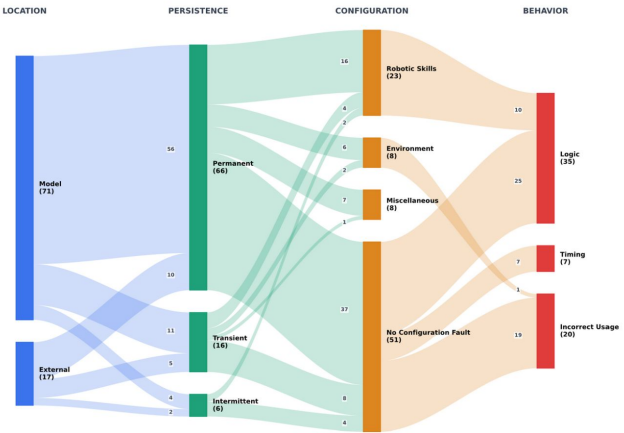
Fault Distribution

- Most faults are Model level (81%)
- Most faults were also Permanent (76%)
- Behavioral faults were caused by:
 - Logic (39%)
 - Incorrect Usage (23%)
 - Timing (8%)
- Configuration Faults were caused by:
 - Robotic Skills (26%)
 - Environment (9%)
 - Others (9%)



Relationship between faults

- Most model faults were also Permanent (80%)
- Most faults were related to behavioral faults (58%)
- All timing faults were unrelated to configuration
- Almost all incorrect usage faults were related to behavioral faults
- Logic faults had the highest correlation with configuration faults



Main Takeaways

Modelling Behavior Trees:

The predominance of logic and incorrect usage faults indicate that, while BTs are expressive, their semantics are not always intuitive to developers

Lack of automated tooling:

We found no automated tools for testing or verification being consistently employed by developers

Software Engineering Practices:

Behavior Trees are used as modeling artifacts but are rarely treated with the same rigor as traditional software artifacts in terms of validation, testing, or lifecycle integration

Thank You!

